Table of Contents

Part I — User Guide (Records Staff)

1. Welcome
2. Signing In
3. Dashboard Overview
4. Searching for Records
5. Managing Records Requests
6. Reviewing Exemption Flags
7. Response Letters
8. Timeline & Messaging
9. Fee Tracking
10. Department Access
11. Quick Reference

Part II — IT Administration Guide

1. System Architecture
2. Hardware Requirements
3. Installation
4. Configuration Reference
5. User & Department Administration
6. Data Source Management

7. Model Management

8. Exemption Rules Administration

9. Compliance & Audit

10. Security

11. Backup & Recovery

12. Monitoring & Troubleshooting

13. Upgrading

Appendices

A. API Endpoint Reference

B. Database Schema Overview

C. Supported Platforms

Part I: User Guide

For City Clerks, Records Officers, Department Liaisons, and City Managers

1. Welcome

What CivicRecords AI Does

CivicRecords AI is a tool that helps your city respond to public records requests faster, more consistently, and with greater confidence. When a resident submits a records request, the system searches across your city's record sources — meeting minutes, resolutions, ordinances, permits, and more — and presents the most relevant results in seconds. It also suggests which exemptions might apply, drafts response letters, and tracks every request from receipt to fulfillment.

What CivicRecords AI Is Not

CivicRecords AI is an assistant, not a decision-maker. Everything it does is a recommendation for you to review. The system suggests and drafts; you review and approve. Every exemption flag, every response letter, and every release decision requires a human sign-off.

It is not a public portal. Residents do not interact with CivicRecords AI directly. You use it internally to process their requests.

Privacy by Design

CivicRecords AI runs entirely on your city's local network or on-premises servers. No records or request data leave your infrastructure. The AI models run locally — no cloud AI services are used.

> **NOTE**
>
> This manual is written for city staff members who use CivicRecords AI day to day: city clerks who process records requests, records officers who manage compliance, department liaisons who search for and provide departmental records, and city managers who oversee the process. You do not need any technical background to use the system.

Who This Manual Is For

Role	Primary Tasks
City Clerk	Receive and process incoming records requests, search for responsive documents, draft and send response letters
Records Officer	Manage compliance, review exemption flags, approve response letters, oversee request workflows
Department Liaison	Search for records within your department, attach documents to requests, provide context for department-specific records
City Manager	Monitor dashboard for workload and compliance, review reports, oversee department performance

2. Signing In

How to Access CivicRecords AI

Open your web browser (Chrome, Edge, or Firefox are recommended) and navigate to the address provided by your IT department. In most installations, this is:

```
http://localhost:8080
```

If your city has deployed CivicRecords AI on a shared network server, your IT administrator will provide the correct address (for example, `http://records.yourcity.local:8080`).

Logging In

1. Enter your **email address** in the Email field.
2. Enter your **password** in the Password field.
3. Select **Sign In**.

Your account is created by your IT administrator. If you do not have an account, contact your IT department.

> **TIP**
>
> If you see a "connection refused" or "page cannot be displayed" error, the CivicRecords AI server may not be running. Contact your IT administrator.

Logging Out

Select your name or initials in the upper-right corner of any page, then select **Sign Out**. Your session expires automatically after a period of inactivity (configured by your IT administrator, typically 30 minutes).

Password Changes

Password management is handled by your IT administrator. If you need to change your password, contact them directly. Self-service password reset is not available in the current version.

3. Dashboard Overview

The Main Dashboard

After signing in, you land on the Dashboard. This is your home screen and provides an at-a-glance view of your workload and the overall status of records requests in the system.

Dashboard Components

Component	Description
Open Requests Counter	The total number of records requests currently in progress (not yet closed or fulfilled).
Approaching Deadlines	Requests with statutory deadlines within the next 3 business days, highlighted in amber or red depending on urgency.
Status Distribution	A breakdown of all active requests by their current status (received, searching, in review, etc.).
Recent Activity	A timeline of the most recent actions across all requests: status changes, documents attached, letters sent.
My Assigned Requests	Requests specifically assigned to you. This is your personal work queue.

> **TIP**
>
> Check the dashboard at the start of each workday to identify urgent deadlines and new assignments.

Navigation Sidebar

The left sidebar provides links to the main areas of the application:

- **Dashboard** — Return to the main overview.
- **Requests** — View and manage all records requests.
- **Search** — Search across all ingested documents.

- **Documents** — Browse ingested documents by source.
- **Reports** — Compliance and workload reports.

4. Searching for Records

How Search Works

CivicRecords AI uses hybrid search that combines two techniques: **keyword search** (traditional full-text matching) and **semantic search** (AI-powered meaning-based retrieval). This means you can search using natural language questions, not just exact keywords.

Performing a Search

1. Select **Search** from the sidebar or use the search bar at the top of any page.
2. Type your search query. You can use natural language (e.g., "building permits issued in 2024 for Main Street") or keywords (e.g., "permit Main Street 2024").
3. Press **Enter** or select the search button.
4. Results appear ranked by relevance, with the most relevant documents at the top.

Understanding Search Results

Each result shows:

- **Document title** and file type (PDF, DOCX, etc.).
- **Relevance score** — a percentage indicating how closely the document matches your query.
- **Highlighted excerpt** — the portion of the document most relevant to your query, with matching terms highlighted.
- **Source** — the data source and department the document belongs to.
- **Date** — when the document was ingested into the system.

Refining Your Search

If your initial search returns too many or too few results:

- **Be more specific** — Add dates, names, or document types to narrow results.
- **Use natural language** — Instead of "variance," try "zoning variance applications denied in 2023."

- **Filter by department** — Use the department filter to limit results to a specific department's records.
- **Filter by file type** — Narrow to PDFs, spreadsheets, or other specific formats.
- **Filter by date range** — Limit results to documents from a specific time period.

> **NOTE**
>
> Search results are limited to documents within your department unless you have Reviewer or Admin access, which provides cross-department visibility.

Attaching Search Results to a Request

When you find a document that is responsive to a records request:

1. Select the **Attach to Request** button on the search result.
2. Choose the request from the dropdown list or enter the request ID.
3. Optionally add a **relevance note** explaining why this document is responsive.
4. Select **Attach**. The document now appears in the request's document list.

5. Managing Records Requests

Creating a New Request

When a public records request arrives (by email, in person, or through your city's intake process):

1. Select **Requests** from the sidebar, then select **New Request**.
2. Fill in the required fields:
   - **Requester Name** — The name of the person or organization making the request.
   - **Requester Email** — Contact email address.
   - **Description** — The full text of the records request as received.
   - **Statutory Deadline** — The date by which your state's open records law requires a response. The system pre-fills this based on your state's default timeline, but you can adjust it.
3. Select **Create Request**. The request is created with a status of **Received**.

Request Status Workflow

Every request moves through a defined workflow. The following table shows each status and what it means:

Status	Meaning	Who Acts
Received	The request has been logged but not yet assigned or acted upon.	Clerk
Clarification Needed	The request is unclear or incomplete; the requester has been asked for more information.	Clerk
Assigned	The request has been assigned to a specific staff member.	Clerk / Manager
Searching	Staff is actively searching for responsive documents.	Staff
In Review	Documents have been gathered and are being reviewed for exemptions and completeness.	Staff / Reviewer
Ready for Release	Documents have been reviewed and are ready for supervisor approval.	Reviewer

Drafted	A response letter has been drafted and is awaiting review.	Staff
Approved	The response letter and documents have been approved by a reviewer.	Reviewer
Fulfilled / Sent	The response has been sent to the requester.	Clerk
Rejected	The request was denied (with documented reasons and legal basis).	Reviewer
Closed	The request is complete and archived.	Clerk

Moving a Request Through the Workflow

To advance a request to its next status:

1. Open the request from the Requests list.
2. The available workflow actions appear as buttons in the sidebar (e.g., "Start Searching," "Submit for Review," "Approve").
3. Select the appropriate action. The system validates the transition and updates the status.
4. A timeline entry is automatically created recording the status change, your identity, and a timestamp.

> **WARNING**
>
> You cannot skip statuses. The system enforces the defined workflow transitions. For example, you cannot move a request directly from "Received" to "Approved" — it must pass through the intermediate steps.

Assigning Requests

Clerks and managers can assign requests to specific staff members:

1. Open the request.
2. Select **Assign** and choose the staff member from the dropdown.
3. The assignee is notified and the request appears in their "My Assigned Requests" dashboard list.

6. Reviewing Exemption Flags

What Exemption Flags Are

When documents are attached to a request, CivicRecords AI automatically scans them for content that may be exempt from disclosure under your state's open records law. These are presented as **exemption flags** — recommendations for your review, not automatic decisions.

Viewing Exemption Flags

Open a request and select the **Exemptions** tab. For each flag, you will see:

- The **highlighted text** that the system flagged.
- The **exemption category** (for example, "Social Security Number" or "Attorney-Client Privilege").
- The **state statute** that may apply.
- A **confidence score** showing how certain the AI is about this flag.

Exemption Categories

Category	Description	Examples
PII (Personal Information)	Personally identifiable information that may need to be redacted for privacy.	Social Security numbers, driver's license numbers, personal phone numbers, dates of birth, home addresses, medical information
Statutory	Content that may be exempt by specific state statute.	Attorney-client privileged communications, law enforcement investigation records, trade secrets, sealed court records
Deliberative Process	Internal deliberations, draft documents, and policy discussions that may be exempt.	Pre-decisional memoranda, internal policy drafts, staff recommendations before a vote
Security	Information that could pose a security risk if disclosed.	Building security plans, IT infrastructure details, emergency response protocols, access codes

Personnel	Employee records that may be partially or fully exempt.	Performance evaluations, disciplinary records, medical leave records, personal contact information of employees

Understanding Confidence Scores

- **90–100%** — Very high confidence. The AI strongly believes this is exempt (for example, a clearly formatted Social Security number).
- **70–89%** — Moderate confidence. The content likely qualifies, but context matters. Review carefully.
- **Below 70%** — Lower confidence. The system flagged it for caution, but it may not be exempt. Use your judgment.

Acting on Exemption Flags

For each flag, you have three options:

1. **Accept** — You agree this content is exempt. It will be noted for redaction in the response documents.
2. **Reject** — You disagree with the flag. The content will be included in the response as-is.
3. **Escalate** — You are unsure and want a reviewer or city attorney to weigh in.

> **SECURITY**
>
> Never release documents with unreviewed exemption flags. Every flag must be accepted or rejected by a staff member before a response letter can be finalized. The system enforces this requirement.

7. Response Letters

Generating a Response Letter

Once you have gathered all responsive documents and reviewed all exemption flags, you can generate a response letter:

1. Open the request and select the **Response Letter** tab.
2. Select **Generate Draft**. The system creates a draft letter using your city's response template, automatically populated with:
   - Requester name and contact information.
   - A description of the records requested.
   - A list of responsive documents being provided.
   - Citations for any exemptions applied, with the corresponding state statutes.
   - Fee information, if applicable.
3. Review the draft carefully. You can edit any part of the letter directly in the editor.
4. Select **Save Draft** when you are satisfied with the letter.

Submitting for Review

After saving a draft:

1. Select **Submit for Review**. The request status changes to **In Review**.
2. A reviewer (Records Officer or authorized supervisor) is notified.
3. The reviewer can:
   - **Approve** — The letter is approved for release. The status changes to **Approved**.
   - **Request Changes** — The reviewer adds comments and returns the letter to the staff member for revision.
   - **Reject** — The draft is rejected entirely. The staff member must create a new draft.

Sending the Response

After a letter is approved:

1. Open the approved request.
2. Select **Mark as Sent** after you have delivered the response through your city's normal channels (email, postal mail, in-person pickup).
3. The request status changes to **Fulfilled / Sent**.

> **NOTE**
>
> CivicRecords AI does not send response letters directly to requesters. You use your city's existing communication channels. The system tracks that the response was sent and records the date and method.

8. Timeline & Messaging

Viewing the Request Timeline

Every request has a **Timeline** tab that shows a chronological log of everything that has happened:

- When the request was created and by whom.
- Every status change, with timestamps and the staff member who made the change.
- Documents that were attached or removed.
- Exemption flags that were reviewed.
- Messages sent to or received from the requester.
- Internal notes added by staff.

The timeline is read-only — you cannot edit or delete entries. This ensures a complete, tamper-proof record of every action taken on the request.

Adding Notes to the Timeline

1. Open the request and go to the **Timeline** tab.
2. Select **Add Note**.
3. Type your note in the text field. You can document research findings, record phone conversations with the requester, or leave instructions for colleagues.
4. Select **Save Note**. The note appears in the timeline with your name and a timestamp.

Sending Messages to Requesters

You can send messages to the requester directly from within a request:

1. Open the request and go to the **Messages** tab.
2. Select **New Message**.
3. Type your message. Common uses include asking for clarification, providing status updates, or notifying the requester about fees.
4. Select **Send**. The message is logged in the timeline and the request's message history.

NOTE

Messages sent through CivicRecords AI are recorded for compliance purposes. Use them for official communications related to the request. The system stores the message content, timestamp, and sender for the audit trail.

9. Fee Tracking

Adding Fees

If your city charges fees for records requests (copying fees, research time, etc.):

1. Open the request and select the **Fees** tab.
2. Select **Add Fee**.
3. Enter the fee type (e.g., "Copying — Standard," "Research Time"), quantity, unit price, and any notes.
4. Select **Save**. The fee appears in the fee table with a running total.

Fee Summary

The Fees tab shows:

- An itemized list of all fees for this request.
- The total amount due.
- Payment status (pending, paid, waived).

Fee information is automatically included in generated response letters when applicable.

> **TIP**
>
> If your city's policy allows fee waivers for certain requesters (media, indigent requesters, etc.), you can mark individual fees as "Waived" with a reason. This is recorded in the audit trail.

10. Department Access

How Department Scoping Works

CivicRecords AI uses department-based access control to ensure staff members see only the records relevant to their work:

- **Staff** and **Read-Only** users can only search documents and view requests within their assigned department.
- **Reviewer** and **Admin** users have cross-department visibility and can view all documents and requests.
- Documents ingested from a data source are associated with the department that owns that data source.

> **NOTE**
>
> If you cannot find a document you expect to see, it may belong to a different department's data source. Contact your Records Officer or a user with cross-department access for assistance.

Department Assignment

Your department assignment is configured by your IT administrator. If you need access to a different department's records, contact your administrator to request a department change or role upgrade.

11. Quick Reference

Glossary

Term	Definition
FOIA / Open Records Request	A formal request from a member of the public for government records. Name varies by state (FOIA, CORA, Public Records Act, etc.).
Responsive Document	A document that contains information relevant to a records request.
Exemption	A legal basis for withholding all or part of a record from public disclosure.
Redaction	The removal or obscuring of exempt information from a document before release.
Statutory Deadline	The legal timeframe within which your city must respond to a records request.
Hybrid Search	A search method that combines keyword matching with AI-powered semantic understanding.
RAG (Retrieval-Augmented Generation)	The AI technique used to find relevant documents and generate contextual responses.
Ingestion	The process of importing documents into CivicRecords AI for indexing and search.
Confidence Score	A percentage indicating how certain the AI is about an exemption flag or search result relevance.
Audit Trail	A tamper-proof log of every action taken in the system, used for compliance verification.

Keyboard Shortcuts

Shortcut	Action
`/` or `Ctrl+K`	Focus the search bar
`Ctrl+Enter`	Submit the current form
`Esc`	Close the current modal or dialog

`?`	Show keyboard shortcuts help

Status Reference

Status	Color	Phase
Received	Blue	Intake
Clarification Needed	Blue	Intake
Assigned	Blue	Intake
Searching	Amber	Processing
In Review	Purple	Review
Ready for Release	Purple	Review
Drafted	Amber	Processing
Approved	Green	Completion
Fulfilled / Sent	Green	Completion
Rejected	Red	Terminal
Closed	Gray	Terminal

Part II: IT Administration Guide

For System Administrators, IT Directors, and DevOps Staff

12. System Architecture

CivicRecords AI is a self-contained, Docker-based application stack designed to run entirely on-premises. The architecture follows a three-tier design: a browser-based client layer, a Python application layer, and a data and AI layer. All components communicate over internal Docker networking with no external cloud dependencies.

12.1 Architecture Overview

The system comprises seven Docker services orchestrated by Docker Compose. The following diagram shows all components, their ports, internal modules, and data flow paths.

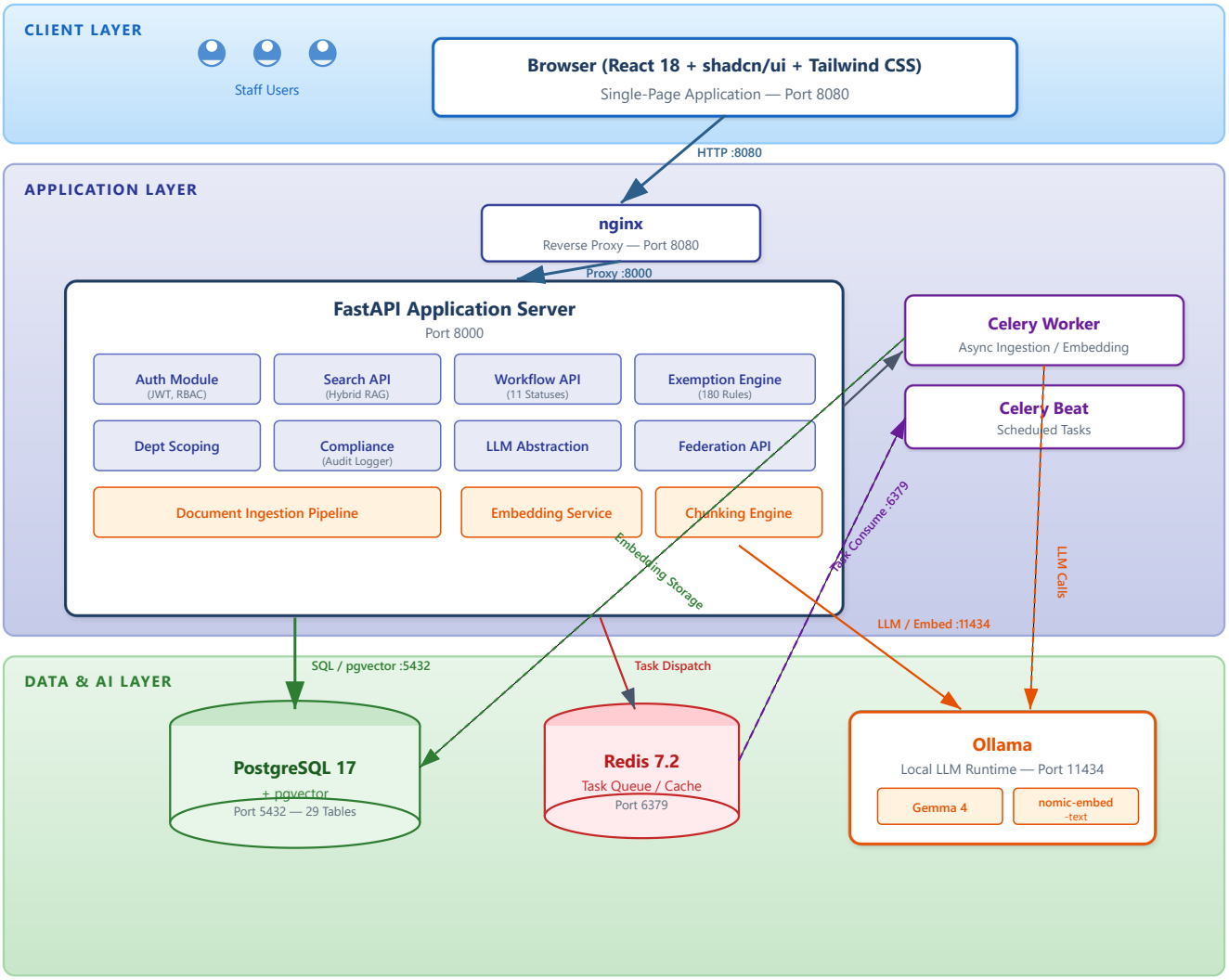

Figure 12-1. CivicRecords AI System Architecture — Docker Compose Stack

12.2 Component Details

Service	Technology	Port	Purpose
Frontend	React 18, shadcn/ui, Tailwind CSS	8080	Single-page application served through nginx
nginx	nginx (latest)	8080	Reverse proxy, static file serving, TLS termination
API	Python 3.12, FastAPI, Uvicorn	8000	Application server with all business logic modules
Worker	Celery 5.x	—	Asynchronous document ingestion, embedding generation, LLM processing

Beat	Celery Beat	—	Scheduled tasks (deadline monitoring, periodic re-indexing)
PostgreSQL	PostgreSQL 17 + pgvector	5432	Primary database (29 tables), vector storage for embeddings
Redis	Redis 7.2	6379	Celery task broker, session cache, rate limiting
Ollama	Ollama (latest)	11434	Local LLM runtime hosting Gemma 4 and nomic-embed-text

12.3 Application Modules

The FastAPI application server contains the following internal modules:

Module	Responsibility
Auth Module	User authentication via JWT tokens, role-based access control (RBAC) with 4 roles: Admin, Staff, Reviewer, Read-Only. Service account management for machine-to-machine API access.
Search API	Hybrid retrieval-augmented generation (RAG) combining semantic vector search (pgvector) with keyword full-text search. Source attribution, session context for iterative refinement.
Workflow API	Records request CRUD operations, status transitions through 11 defined statuses with validation, document association, deadline management.
Exemption Engine	Rules engine with 180 pre-loaded rules (regex patterns, keyword matching, statutory phrase detection) plus LLM-powered suggestion layer. Each exemption category is a separate detector.
Department Scoping	Row-level access control tying users, data sources, documents, and requests to departments. Enforces department boundaries for Staff and Read-Only roles.
Compliance Module	Middleware-based audit logger recording every API call. Hash-chained, append-only audit trail. Exportable as CSV/JSON for external review.
LLM Abstraction	Model-agnostic interface wrapping Ollama. Supports chat completion and embedding endpoints. Swap models without application code changes.
Federation API	REST endpoints for inter-instance communication. Allows one CivicRecords AI installation to query another with scoped, authenticated access via service account API keys.

12.4 Ingestion Pipeline

The document ingestion pipeline processes files from configured data sources through a two-track architecture. Files are routed based on type detection: standard document formats follow a fast track

through native parsers, while scanned documents and images are routed through the LLM-powered track for multimodal text extraction.

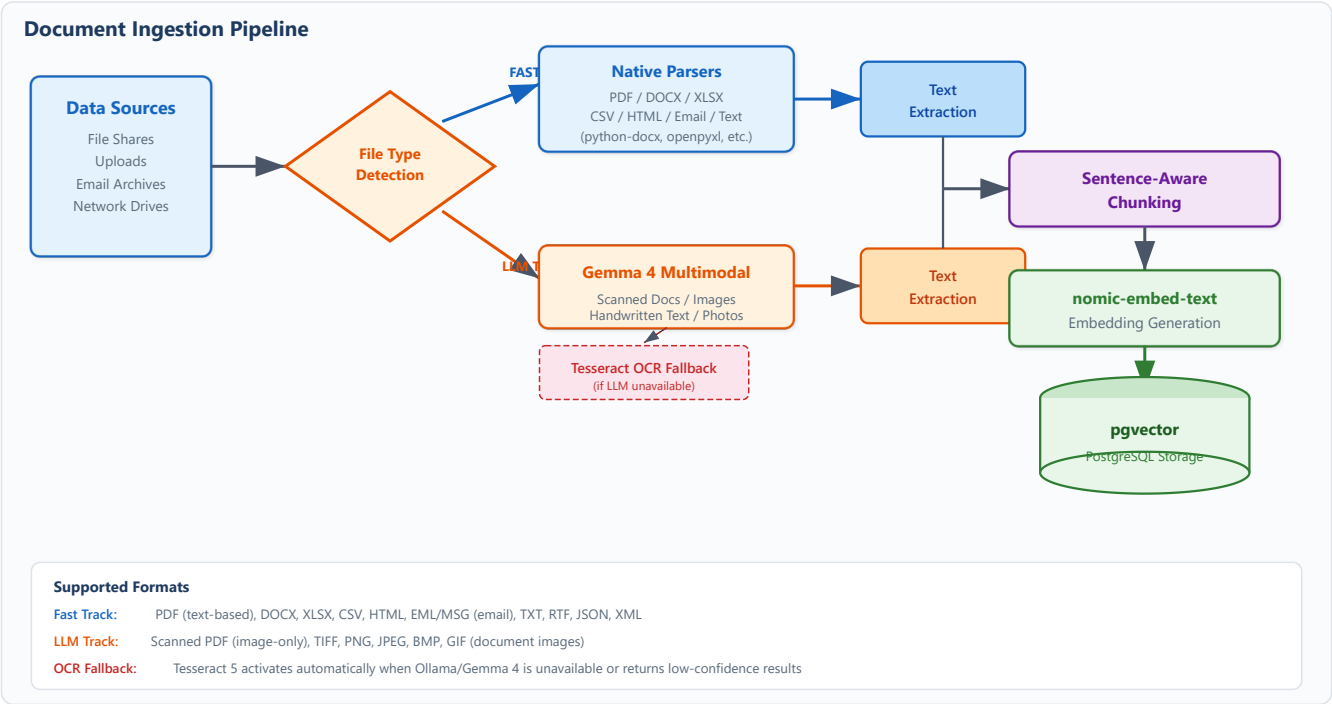

Figure 12-2. Two-Track Document Ingestion Pipeline

12.5 Request Lifecycle Workflow

Records requests move through 11 defined statuses. Status transitions are validated by the workflow engine — invalid transitions are rejected. The following diagram shows all valid transitions, color-coded by processing phase.

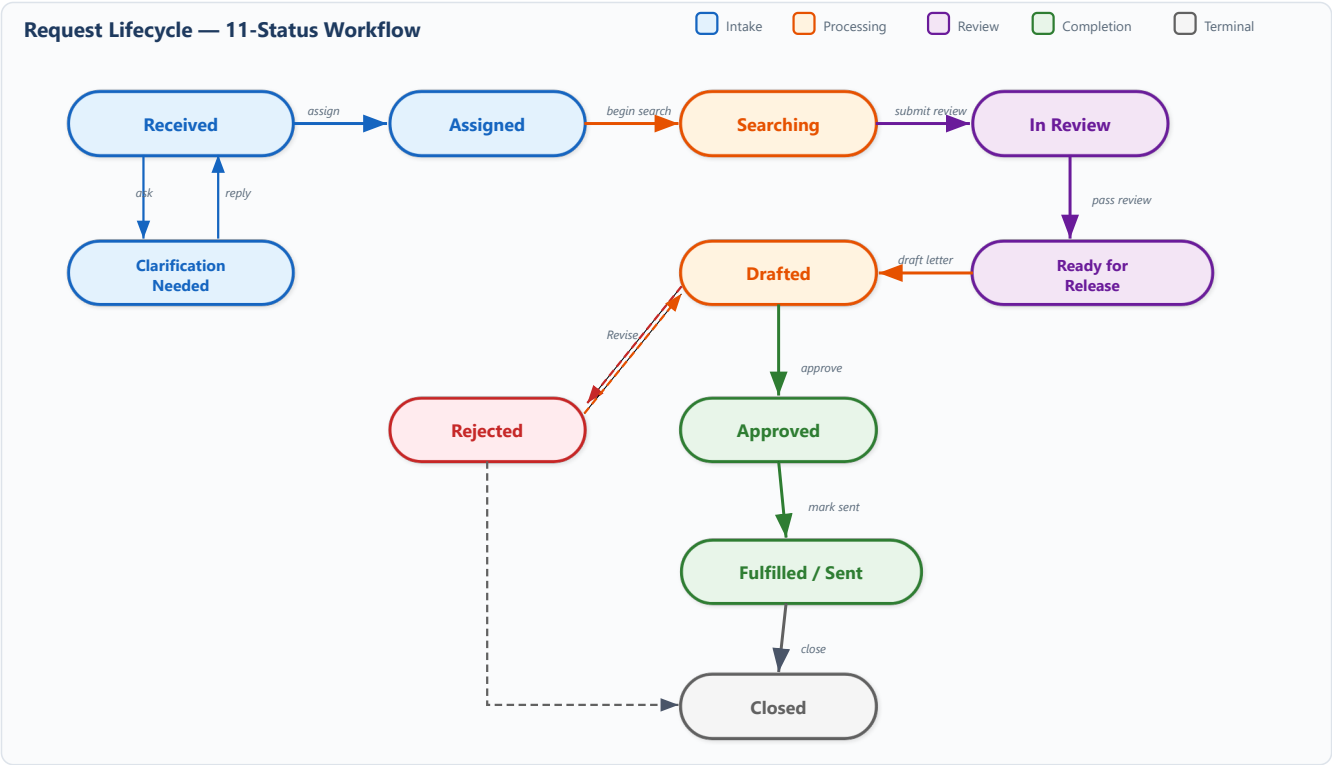

Figure 12-3. Records Request Lifecycle — 11-Status Workflow with Phase Color Coding

12.6 Network Architecture

All services communicate over a single Docker bridge network (`civicrecords-net`). Only the nginx service exposes a port to the host. External access topology:

Service	Host Binding	Internal Only
nginx	0.0.0.0:8080	No (externally accessible)
FastAPI	—	Yes (nginx proxies to :8000)
PostgreSQL	—	Yes (:5432 internal)
Redis	—	Yes (:6379 internal)
Ollama	—	Yes (:11434 internal)
Celery Worker	—	Yes (no port)
Celery Beat	—	Yes (no port)

SECURITY

The system binds to `localhost` or a city-designated internal IP only. It must never be exposed to the public internet. HTTPS with self-signed or city-provided TLS certificates is recommended for all web interfaces.

13. Hardware Requirements

Server Specifications

Component	Minimum Spec	Recommended Spec
CPU	AMD Ryzen 7 (8-core)	AMD Ryzen 9 (12-16 core)
RAM	32 GB DDR4/DDR5	64 GB DDR5
Storage	1 TB NVMe SSD	2 TB NVMe SSD
GPU	Integrated (CPU inference)	Discrete AMD or NVIDIA with 8+ GB VRAM
Network	Gigabit Ethernet	Gigabit Ethernet
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Estimated Cost	~$800	~$1,200

> **NOTE**
>
> The system must deliver usable response times (under 30 seconds for a typical query-and-retrieve cycle) on the minimum spec without a discrete GPU. GPU acceleration is a performance bonus, not a requirement.

Docker Resource Allocation

When running on Windows or macOS with Docker Desktop, allocate at least:

- **Memory:** 32 GB (Docker Desktop → Settings → Resources)
- **CPU:** All available cores
- **Disk:** 50 GB free for Docker images, volumes, and ingested data

GPU Acceleration

GPU acceleration is optional but significantly improves LLM inference speed:

GPU Type	Driver Requirement	Docker Compose Overlay
NVIDIA	CUDA drivers + NVIDIA Container Toolkit	`docker-compose.nvidia.yml`
AMD	ROCm drivers	`docker-compose.amd.yml`
Intel / DirectML	DirectML runtime (Windows)	`docker-compose.directml.yml`
None	—	Default (CPU inference)

Physical Deployment

- **Location:** Locked office, server closet, or data center with physical access controls.
- **Power:** Uninterruptible power supply (UPS) recommended.
- **Network:** Static IP address, appropriate firewall rules, internal DNS entry.

14. Installation

Prerequisites

- Docker Engine 24.0+ and Docker Compose v2 (included with Docker Desktop on Windows/macOS)
- At least 32 GB RAM available to Docker
- 50 GB free disk space
- Network access to Docker Hub during initial installation (for image pulls)
- For GPU acceleration: appropriate drivers installed (AMD ROCm, NVIDIA CUDA, or DirectML)

14.1 Windows Installation

Open PowerShell as Administrator in the project directory and run:

```
.\install.ps1
```

The script performs the following steps:

1. Verifies Docker Desktop is installed and the daemon is running
2. Verifies Docker Compose v2 is available
3. Creates `.env` from `.env.example` and generates a cryptographically random JWT secret
4. Pauses for you to configure admin credentials in `.env`
5. Runs hardware detection (`scripts/detect_hardware.ps1`) and writes `.env.hardware`
6. Selects the appropriate Docker Compose overlay (GPU or host Ollama if applicable)
7. Pulls base Docker images and builds application images
8. Starts PostgreSQL and Redis; waits for database readiness (up to 60 seconds)
9. Runs Alembic database migrations
10. Starts all seven services
11. Waits for the API health endpoint to respond
12. Pulls the embedding model (`nomic-embed-text`)

13. Prints the recommended language model command and access URLs

14.2 Linux Installation (Ubuntu/Debian)

Run from the project directory:

```
chmod +x install.sh
./install.sh
```

The Linux installer follows the same steps as the Windows installer but uses `scripts/detect_hardware.sh` for hardware detection and operates natively without Docker Desktop.

14.3 macOS Installation

Run from the project directory:

```
chmod +x install.sh
./install.sh
```

The macOS installer uses Docker Desktop and follows the same workflow. GPU acceleration is not available on macOS; the system uses CPU inference only.

> **WARNING**
>
> Do not skip the hardware detection step. The installer uses this information to select the correct Docker Compose overlay for your hardware. Running with the wrong overlay may cause the Ollama container to fail to start.

14.4 Post-Installation Verification

After installation completes, verify all services are healthy:

```
# Check all containers are running
docker compose ps
```

```
# Verify API health
curl http://localhost:8000/health

# Verify frontend is accessible
curl -I http://localhost:8080

# Check Ollama models
docker compose exec ollama ollama list
```

14.5 Pulling the Language Model

The embedding model (`nomic-embed-text`) is pulled automatically during installation. The language model must be pulled separately:

```
docker compose exec ollama ollama pull gemma3:4b
```

> **TIP**
>
> For systems with limited resources, the 4-billion parameter model (`gemma3:4b`) provides the best balance of quality and speed. Systems with 16+ GB VRAM can use larger variants.

15. Configuration Reference

Environment Variables

All configuration is managed through the `.env` file in the project root. The following table documents all available variables:

Variable	Default	Description
`DATABASE_URL`	`postgresql+asyncpg://civicrecords:...@postgres:5432/civicrecords`	PostgreSQL connection string
`REDIS_URL`	`redis://redis:6379/0`	Redis connection string
`JWT_SECRET`	*(auto-generated)*	Secret key for JWT token signing. Must be a cryptographically random string.
`JWT_LIFETIME_SECONDS`	`3600`	JWT token expiration in seconds (default: 1 hour)
`ADMIN_EMAIL`	—	Email address for the initial admin account
`ADMIN_PASSWORD`	—	Password for the initial admin account. Change after first login.
`OLLAMA_BASE_URL`	`http://ollama:11434`	Ollama API endpoint

`LLM_MODEL`	`gemma3:4b`	Default language model for chat completions
`EMBEDDING_MODEL`	`nomic-embed-text`	Model used for document embedding
`CHUNK_SIZE`	`512`	Target chunk size in tokens for document splitting
`CHUNK_OVERLAP`	`50`	Overlap in tokens between adjacent chunks
`MAX_SEARCH_RESULTS`	`20`	Maximum number of search results to return per query
`SIMILARITY_THRESHOLD`	`0.3`	Minimum cosine similarity score for vector search results
`LOG_LEVEL`	`INFO`	Application log level (DEBUG, INFO, WARNING, ERROR)
`CORS_ORIGINS`	`http://localhost:8080`	Allowed CORS origins (comma-separated)

SECURITY

Never commit the `.env` file to version control. The `.gitignore` file excludes it by default. Store a backup of your production `.env` in a secure location (e.g., password manager, encrypted volume).

16. User & Department Administration

Role Hierarchy

Role	Level	Permissions
Admin	4	Full access: manage users, departments, data sources, models, exemption rules, system settings. Cross-department visibility.
Reviewer	3	Approve/reject response letters, review exemption flags, cross-department visibility.
Staff	2	Create/manage requests, search, attach documents, draft letters. Limited to assigned department.
Read-Only	1	View requests and documents within assigned department. Cannot create or modify.

Creating Users

```
curl -X POST http://localhost:8000/auth/register \
  -H "Authorization: Bearer <admin-jwt-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "jsmith@yourcity.gov",
    "password": "temporary-password-123",
    "role": "staff",
    "department_id": "dept-uuid-here"
  }'
```

> **TIP**
>
> Instruct new users to change their password after first login. Self-service password reset is planned for a future release.

Department Scoping Behavior

- Staff and Read-Only users can only search documents and view requests within their assigned department.
- Reviewer and Admin users have cross-department visibility.
- Documents ingested from a data source are associated with the department that owns that data source.
- If a user has no department assigned, they cannot see any department-scoped content (except Admins, who see everything).

Service Accounts for Federation

Service accounts enable machine-to-machine API access for integrations with other municipal systems (e.g., a records management system pushing documents). Service accounts authenticate with API keys rather than JWT tokens.

```
curl -X POST http://localhost:8000/service-accounts/ \
  -H "Authorization: Bearer <admin-jwt-token>" \
  -H "Content-Type: application/json" \
  -d '{"name": "RMS Integration", "description": "Automated document feed from Records Managem
```

> **SECURITY**
>
> The API key is returned only once at creation time. It is stored as a bcrypt hash in the database and cannot be retrieved. Record it securely immediately. If lost, create a new service account.

17. Data Source Management

Adding File Directory Sources

A directory data source points to a file system path accessible to the API container. This path must be mounted as a Docker volume.

```
curl -X POST http://localhost:8000/datasources/ \
  -H "Authorization: Bearer <admin-jwt-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "City Council Minutes",
    "source_type": "directory",
    "connection_config": {"path": "/data/council-minutes"},
    "department_id": "dept-uuid-here"
  }'
```

Volume Mounting

Add the source directory as a volume mount in `docker-compose.override.yml`:

```
services:
  api:
    volumes:
      - /path/on/host/council-minutes:/data/council-minutes:ro
  worker:
    volumes:
      - /path/on/host/council-minutes:/data/council-minutes:ro
```

> **NOTE**
>
> Both the API and Worker containers need access to the data source path. Mount as read-only (`:ro`) when possible.

Triggering Ingestion

```
# Trigger ingestion for a specific data source
curl -X POST http://localhost:8000/datasources/{source_id}/ingest \
  -H "Authorization: Bearer <admin-jwt-token>"

# Check ingestion status
curl http://localhost:8000/datasources/{source_id}/status \
  -H "Authorization: Bearer <admin-jwt-token>"
```

Incremental Ingestion

CivicRecords AI tracks file modification timestamps. Subsequent ingestion runs only process new or modified files. Deleted files are marked as inactive but remain in the database for audit purposes.

Supported File Formats

Track	Formats
Fast Track (native parsers)	PDF (text-based), DOCX, XLSX, CSV, HTML, EML, MSG, TXT, RTF, JSON, XML
LLM Track (Gemma 4 multimodal)	Scanned PDF, TIFF, PNG, JPEG, BMP, GIF

18. Model Management

Listing Available Models

```
docker compose exec ollama ollama list
```

Pulling New Models

```
# Pull a language model
docker compose exec ollama ollama pull gemma3:4b

# Pull a larger model (requires more VRAM/RAM)
docker compose exec ollama ollama pull gemma3:12b

# Verify the model is available
docker compose exec ollama ollama list
```

Switching the Active Model

Update the `LLM_MODEL` variable in `.env` and restart the API service:

```
# Edit .env: LLM_MODEL=gemma3:12b
docker compose restart api worker
```

Model Recommendations by Hardware

Hardware Tier	Recommended Model	VRAM / RAM Needed
CPU-only, 32 GB RAM	`gemma3:4b`	~8 GB RAM

GPU with 8 GB VRAM	`gemma3:4b`	~4 GB VRAM
GPU with 12+ GB VRAM	`gemma3:12b`	~10 GB VRAM
GPU with 24+ GB VRAM	`gemma3:27b`	~20 GB VRAM

> **WARNING**
>
> Do not remove the `nomic-embed-text` model. It is required for document embedding and search functionality. Removing it will break search until it is re-pulled.

19. Exemption Rules Administration

Rules Overview

CivicRecords AI ships with approximately 180 pre-loaded exemption rules covering common exemption categories from all 50 state open records statutes. Each rule uses one of two detection methods:

- **Regex:** Regular expression pattern matching (e.g., Social Security number format `\d{3}-\d{2}-\d{4}`)
- **Keyword:** Keyword and phrase matching (e.g., "attorney-client privilege," "trade secret")

Managing Rules via API

```
# List all rules
curl http://localhost:8000/exemptions/rules/ \
  -H "Authorization: Bearer <admin-jwt-token>"

# Create a new rule
curl -X POST http://localhost:8000/exemptions/rules/ \
  -H "Authorization: Bearer <admin-jwt-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "state_code": "CO",
    "category": "personnel",
    "rule_type": "keyword",
    "rule_definition": "performance evaluation"
  }'

# Disable a rule
curl -X PATCH http://localhost:8000/exemptions/rules/{rule_id} \
  -H "Authorization: Bearer <admin-jwt-token>" \
  -H "Content-Type: application/json" \
  -d '{"enabled": false}'
```

Exemption Dashboard

The admin panel provides an Exemptions dashboard showing:

- Total flags generated, broken down by status (accepted, rejected, pending).
- Flag counts by category.
- Acceptance rate percentage.
- Active rules versus total rules.

Use this dashboard to identify rules that are generating too many false positives (low acceptance rate) and consider disabling or refining them.

> **TIP**
>
> All changes to exemption rules are logged in the audit trail. This includes rule creation, modification, enabling, and disabling. The audit log records which admin made the change and when.

20. Compliance & Audit

Audit Trail Architecture

CivicRecords AI maintains a hash-chained, append-only audit log. Every API call is logged with:

- Timestamp (UTC)
- User identity (email, role)
- Action performed (HTTP method, endpoint, parameters)
- Result (success/failure, status code)
- Hash of the previous entry (chain integrity)

Exporting Audit Logs

```
# Export audit logs as JSON
curl "http://localhost:8000/admin/audit-logs?format=json&start_date=2026-01-01&end_date=2026-0
  -H "Authorization: Bearer <admin-jwt-token>" \
  -o audit-export.json

# Export as CSV
curl "http://localhost:8000/admin/audit-logs?format=csv&start_date=2026-01-01&end_date=2026-03
  -H "Authorization: Bearer <admin-jwt-token>" \
  -o audit-export.csv
```

Compliance Reporting

The system provides several built-in compliance reports:

Report	Description
Response Time Report	Average and median response times, broken down by department and time period. Identifies requests that exceeded statutory deadlines.

Exemption Accuracy Report	Flag acceptance/rejection rates by category, department, and time period. Helps assess whether AI flagging is systematically biased.
Workload Distribution	Request counts and processing times by staff member and department.
Disclosure Report	Summary of records released, withheld, and partially redacted for a given period.

Exemption Detection Auditability

The exemption engine is designed for transparency and external review:

- Dashboard showing flag acceptance/rejection rates by category, department, and time period.
- Export flag accuracy data for external review.
- Configuration interface for adjusting exemption rules without code changes, with all changes logged.
- Documentation of exemption rule sources with version tracking.

21. Security

Authentication

- **JWT tokens** for user authentication with configurable expiration.
- **bcrypt** password hashing with automatic salt generation.
- **API keys** for service accounts (machine-to-machine), stored as bcrypt hashes.
- Session expiration after configurable inactivity period.

Authorization

- Role-based access control (RBAC) with 4 roles: Admin (level 4), Reviewer (3), Staff (2), Read-Only (1).
- Every API endpoint requires a minimum role level; the `require_role` dependency checks the user's role against the hierarchy.
- Department-scoped row-level access control for Staff and Read-Only roles.

Data Security

- All data stored in local PostgreSQL with filesystem-level encryption (LUKS, configured at OS level).
- No plaintext credential storage — secrets managed via environment variables.
- Audit logs are append-only and tamper-evident (hash-chained).

Network Security

- System binds to `localhost` or a city-designated internal IP only.
- HTTPS with self-signed or city-provided TLS certificates for all web interfaces.
- All outbound traffic is logged, allowlisted, and auditable.
- No external cloud AI services — all LLM processing is local.

> **SECURITY**

> CivicRecords AI processes potentially sensitive government records. Ensure that the server is physically secured, network access is restricted to authorized personnel, and all security patches for the host operating system are applied promptly.

22. Backup & Recovery

Database Backup

```
# Full database backup (compressed custom format)
docker compose exec -T postgres pg_dump -U civicrecords -Fc civicrecords > backup-$(date +%Y%m

# Backup with timestamp
docker compose exec -T postgres pg_dump -U civicrecords -Fc civicrecords > civicrecords-backup
```

Restoring from Backup

```
# Stop the application (keep database running)
docker compose stop api worker beat

# Restore the database
docker compose exec -T postgres pg_restore -U civicrecords -d civicrecords --clean --if-exists

# Restart services
docker compose up -d
```

Configuration Backup

Back up these files regularly:

File	Contents
`.env`	All environment configuration, including JWT secret
`.env.hardware`	Hardware detection results
`docker-compose.override.yml`	Custom volume mounts and service overrides

Backup Schedule Recommendations

Frequency	What to Back Up	Retention
Daily	PostgreSQL database dump	30 days
Weekly	Full system backup (database + config + volumes)	12 weeks
Before upgrades	Full database dump + config files	Until upgrade verified

> **WARNING**
>
> Always back up the database before performing any upgrade. Database migrations are forward-only and cannot be easily reversed without a backup.

23. Monitoring & Troubleshooting

Health Checks

```
# API health endpoint
curl http://localhost:8000/health

# Individual service checks
docker compose exec postgres pg_isready -U civicrecords
docker compose exec redis redis-cli ping
docker compose exec ollama ollama list
```

Admin Status Dashboard

The admin panel (`/admin`) displays a system status page showing:

- **Database:** Connection status (green/red indicator)
- **Ollama:** LLM runtime status and loaded models
- **Redis:** Task queue status
- **User Count:** Total registered users
- **Audit Log Count:** Total audit entries
- **Version:** Current application version

Viewing Logs

```
# All services
docker compose logs -f

# Specific service
docker compose logs -f api
docker compose logs -f worker
```

```
# Last 100 lines
docker compose logs --tail=100 api
```

Common Issues

Symptom	Likely Cause	Resolution
"Connection refused" on port 8080	nginx or API not running	`docker compose up -d` and check logs
Search returns no results	No documents ingested, or embedding model missing	Verify data sources and run `ollama list` to check for `nomic-embed-text`
Slow LLM responses (>60s)	Insufficient RAM or CPU for the selected model	Switch to a smaller model or add GPU acceleration
Celery tasks stuck in "pending"	Redis connection issue or worker not running	Check Redis connectivity and restart worker: `docker compose restart worker`
Database migration errors	Schema conflict from skipped version	Restore from backup and upgrade sequentially through each version
Ollama "model not found"	Model not pulled	Run `docker compose exec ollama ollama pull <model>`
"Permission denied" on data source	Volume mount permissions	Verify the container user has read access to the mounted directory

Performance Monitoring

```
# Container resource usage
docker stats

# Database size
docker compose exec postgres psql -U civicrecords -c "SELECT pg_size_pretty(pg_database_size('

# Vector index size
docker compose exec postgres psql -U civicrecords -c "SELECT pg_size_pretty(pg_total_relation_
```

24. Upgrading

Upgrade Procedure

1. Back up the database before any upgrade:

```
docker compose exec -T postgres pg_dump -U civicrecords -Fc civicrecords > pre-upgrade-back
```

2. Pull the latest code:

```
git pull origin main
```

3. Rebuild Docker images:

```
docker compose build
```

4. Run database migrations:

```
docker compose run --rm api alembic upgrade head
```

5. Restart all services:

```
docker compose up -d
```

6. Verify health:

```
curl http://localhost:8000/health
```

> **TIP**
>
> The API service automatically runs Alembic migrations on startup. However, running migrations explicitly as a separate step (step 4) allows you to catch migration errors before bringing the full stack online.

Database Migrations

CivicRecords AI uses Alembic for database schema migrations. Migration files are stored in `backend/alembic/versions/`. Key commands:

```
# Apply all pending migrations
docker compose exec api alembic upgrade head

# Check current migration version
docker compose exec api alembic current

# View migration history
docker compose exec api alembic history
```

Version Compatibility

Always upgrade sequentially through minor versions. Skipping major versions may result in migration conflicts. Check the `CHANGELOG.md` for breaking changes between versions.

Rollback Procedure

1. Stop all services: `docker compose down`
2. Restore the database from backup:

   ```
   docker compose up -d postgres
   docker compose exec -T postgres pg_restore -U civicrecords -d civicrecords --clean --if-ex
   ```

3. Check out the previous version: `git checkout v1.0.0`
4. Rebuild and restart: `docker compose build && docker compose up -d`

Appendices

A. API Endpoint Reference

Authentication

Method	Endpoint	Min Role	Description
POST	`/auth/login`	—	Authenticate and receive JWT token
POST	`/auth/register`	Admin	Create a new user account
GET	`/users/me`	Read-Only	Get current user profile

Records Requests

Method	Endpoint	Min Role	Description
GET	`/requests/`	Staff	List all requests (department-scoped)
POST	`/requests/`	Staff	Create a new request
GET	`/requests/{id}`	Staff	Get request details
PATCH	`/requests/{id}`	Staff	Update request fields or status
GET	`/requests/{id}/documents`	Staff	List documents attached to request
POST	`/requests/{id}/documents`	Staff	Attach a document to request
POST	`/requests/{id}/submit-review`	Staff	Submit request for review
POST	`/requests/{id}/approve`	Reviewer	Approve request for release
POST	`/requests/{id}/reject`	Reviewer	Reject request
GET	`/requests/{id}/timeline`	Staff	Get request timeline events
POST	`/requests/{id}/timeline`	Staff	Add a timeline event / note

GET	`/requests/{id}/messages`	Staff	Get request messages
POST	`/requests/{id}/messages`	Staff	Send a message on request
GET	`/requests/{id}/fees`	Staff	Get request fee items
POST	`/requests/{id}/fees`	Staff	Add a fee item
POST	`/requests/{id}/response-letter`	Staff	Generate a response letter draft
GET	`/requests/{id}/response-letter`	Staff	Get current response letter
PATCH	`/requests/{id}/response-letter/{letter_id}`	Staff	Update response letter (approve requires Reviewer)

Search

Method	Endpoint	Min Role	Description
POST	`/search/`	Staff	Hybrid search across ingested documents
GET	`/search/history`	Staff	Get search history for current user

Documents & Data Sources

Method	Endpoint	Min Role	Description
GET	`/documents/`	Staff	List ingested documents
GET	`/documents/{id}`	Staff	Get document details and metadata
GET	`/datasources/`	Admin	List configured data sources
POST	`/datasources/`	Admin	Add a new data source
POST	`/datasources/{id}/ingest`	Admin	Trigger ingestion for data source
GET	`/datasources/stats`	Admin	Get ingestion statistics

Exemptions

Method	Endpoint	Min Role	Description
GET	`/exemptions/rules/`	Admin	List all exemption rules
POST	`/exemptions/rules/`	Admin	Create a new exemption rule
PATCH	`/exemptions/rules/{id}`	Admin	Update/toggle an exemption rule
GET	`/exemptions/dashboard`	Admin	Get exemption dashboard statistics

Administration

Method	Endpoint	Min Role	Description
GET	`/admin/status`	Admin	System health and service status
GET	`/admin/audit-logs`	Admin	Query and export audit logs
GET	`/health`	—	Unauthenticated health check endpoint
POST	`/service-accounts/`	Admin	Create a service account

B. Database Schema Overview

CivicRecords AI uses PostgreSQL 17 with the pgvector extension for vector storage. The schema consists of 29 tables organized into the following functional groups:

Authentication & Administration

Table	Description
`users`	User accounts with role enum (admin, staff, reviewer, read_only), department assignment, and fastapi-users fields
`service_accounts`	Machine-to-machine API key accounts with roles and bcrypt-hashed keys
`departments`	Department definitions for access scoping

Documents & Ingestion

Table	Description
`data_sources`	Configured data source connections (directory paths, types, department ownership)
`documents`	Ingested document metadata (filename, file type, size, ingestion status, error messages)
`document_chunks`	Text chunks extracted from documents (content, position, chunk index)
`document_embeddings`	Vector embeddings for document chunks (pgvector column, model identifier)

Records Requests

Table	Description
`records_requests`	Public records requests with status workflow, requester info, deadlines, and assignment
`request_documents`	Many-to-many associations between requests and documents with inclusion decisions and relevance notes

`document_cache`	Cached document files per request for legal defensibility
`request_timeline`	Chronological event log for each request (status changes, notes, actions)
`request_messages`	Messages between staff and requesters
`request_fees`	Fee items associated with requests (type, amount, status)
`response_letters`	Draft and final response letters with approval status

Exemptions

Table	Description
`exemption_rules`	Configurable rules (state code, category, rule type, definition, enabled flag)
`exemption_flags`	Flags generated by the exemption engine (document, chunk, category, confidence, status)

Search & Analytics

Table	Description
`search_sessions`	Search session context for iterative refinement
`search_queries`	Individual search queries with result counts and timing

Audit & Compliance

Table	Description
`audit_log`	Hash-chained, append-only audit trail of all system actions

C. Supported Platforms

Operating Systems

Platform	Support Level	Notes
Ubuntu 24.04 LTS	Primary (recommended)	Full support including GPU acceleration
Debian 12 (Bookworm)	Primary	Full support including GPU acceleration
Windows 11 Pro/Enterprise	Supported	Requires Docker Desktop. GPU via CUDA (NVIDIA) or DirectML.
Windows 10 (21H2+)	Supported	Requires Docker Desktop with WSL 2 backend
macOS 13+ (Ventura)	Supported	Docker Desktop required. CPU inference only (no GPU acceleration).
RHEL 9 / Rocky Linux 9	Community	Should work with Docker CE; not formally tested

Browsers

Browser	Minimum Version
Google Chrome	120+
Microsoft Edge	120+
Mozilla Firefox	120+
Safari	17+

Docker Requirements

Component	Minimum Version

Docker Engine	24.0
Docker Compose	v2.20
Docker Desktop (Windows/macOS)	4.25

© 2026 CivicRecords AI Project — Unified Operations Manual v1.1.0

AI-Powered Open Records Support for American Cities

This document is confidential and intended for authorized municipal staff and IT personnel only.